Light ABFT: Low-Cost Error Detection for Matrix Multiplication on an FPGA Systolic Array

Reproduction and evaluation of F. Libano, P. Rech and J. Brunhaver, *Efficient Error Detection for Matrix Multiplication With Systolic Arrays on FPGAs*, IEEE Transactions on Computers, vol. 72, no. 8, 2023

VLSI Course Project

Student	Roll number
Naseeb	S20240020315
S K Dhanvitha	S20240020328

Under the guidance of **Prof. Ramesh Kumar**

Indian Institute of Information Technology, Sri City

Hardware: EDGE Artix-7 FPGA board (XC7A35T-1FTG256) · Tools: AMD Vivado 2024.2, Python 3

November 2026

Contents

1. Introduction and main idea of the paper
2. Background theory
3. Project scope and paper coverage
4. Hardware design
5. How to reproduce this project (step by step)
6. Hardware results: simulation, board, Vivado reports
7. Fault-injection campaign (Experiment 1)
8. Cost analysis (Experiments 2, 3 and 5)
9. Effect on a neural network (Experiment 4)
10. Paper vs. ours: summary comparison
11. Our contribution
12. Limitations and future work
13. Conclusion

References

1. Introduction and main idea of the paper

Matrix multiplication is the core operation of signal processing, image processing, scientific computing and neural networks. FPGAs accelerate it efficiently with **systolic arrays**: grids of small multiply-accumulate units that pass data to their neighbours every clock cycle.

SRAM-based FPGAs have a specific weakness. The circuit itself is stored in configuration memory, and a radiation particle can flip one of those bits. Unlike in a CPU, where such a flip usually corrupts one value once, in an FPGA the **circuit stays broken until the FPGA is reprogrammed**, so every later result may be wrong. The system therefore needs a fast, cheap way to notice that results have gone wrong, so it can trigger reconfiguration.

Main idea of the paper in three steps

1. **Study the fault model.** The authors injected over 618,000 configuration faults into a 4×4 systolic array. Because each processing element (PE) contributes to several outputs, a single fault usually corrupts a whole row or column of the result: about 70% of errors were line errors.
2. **Draw the conclusion.** The classic error-correcting method, Algorithm-Based Fault Tolerance (ABFT, 1984), can only correct single wrong values, and it needs about 2N+1 extra multipliers for an N×N array. Since most FPGA errors are line errors, and the fault must be repaired by reconfiguration anyway, correction capability is mostly wasted.
3. **Propose Light ABFT.** Only *detect* errors, as cheaply as possible: compare the sum of all outputs with one number computed from the inputs. This needs a single extra multiplier. On hardware it detected 97.4% of injected faults, and it adds under 1% arithmetic overhead to real CNNs.

Project objectives

- Implement a 4×4 weight-stationary systolic array and the Light ABFT checker in Verilog.
- Verify the design in simulation and demonstrate fault detection on a physical Artix-7 FPGA board.
- Build a traditional ABFT checker on the same array for a direct cost comparison.
- Reproduce the paper's main experiments (fault patterns, detection rate, cost model, CNN overhead, CNN fault sensitivity) and compare our numbers with the paper's.

2. Background theory

2.1 Radiation faults in FPGAs

Cosmic rays hitting the atmosphere create showers of secondary particles, mostly neutrons, that reach the ground and pass through chips. A particle striking a memory cell can flip the stored bit (a **Single Event Upset**, SEU). In an SRAM-based FPGA the most vulnerable memory is the configuration memory, which defines the logic and wiring, so an SEU can change the circuit. The fault is **persistent**: it remains until the bitstream is reloaded or the memory is scrubbed. If it goes unnoticed, it produces silent data corruption.

2.2 Weight-stationary systolic array

Each processing element (PE) performs one multiply-accumulate per clock cycle: $\mathbf{sum_{out} = sum_{in} + a_{in} \times b}$. In a **weight-stationary** array, PE(z,y) first stores element B[z][y]. Matrix A then enters from the left, one diagonal per cycle (each row delayed by one more cycle, called skewing), A values move right, and partial sums move down. Element C[x][y] of the result

leaves the bottom of column y. All PEs work in parallel, and each input is read from memory once.

Because the partial sums of a whole column pass through every PE in that column, **one faulty PE corrupts many outputs in the same column**. This is why the paper observes mostly line errors.

2.3 Traditional ABFT and Light ABFT

Algorithm-Based Fault Tolerance (ABFT) checks a computation using properties of the algorithm instead of duplicating hardware. Traditional ABFT (Huang and Abraham, 1984) adds a checksum for every row and every column of C. It can locate and correct a single wrong element, but it needs 2N+1 extra multipliers for an N×N array.

Light ABFT keeps only one check. Let $SV_A[z]$ be the sum of column z of A, and $SV_B[z]$ the sum of row z of B. Then:

$$\mathbf{L = \Sigma_z\ SV_A[z] \cdot SV_B[z] = \Sigma_x\ \Sigma_y\ C[x][y]}$$

L is computed from the *inputs*; the sum of C is computed from the *outputs*. If they differ, an error occurred. The proof only uses the distributive law: expanding $SV_A \cdot SV_B$ gives $\Sigma_x\Sigma_y\Sigma_z$ A[x][z]·B[z][y], which is exactly the sum of every element of C.

Worked example (2×2). A = [[1,2],[3,4]], B = [[5,6],[7,8]] gives C = [[19,22],[43,50]], so ΣC = 134. SV_A = [4, 6], SV_B = [11, 15], so L = 4×11 + 6×15 = 134. If a fault turns 19 into 20, ΣC = 135 ≠ 134 and the error is detected.

The trade-off: Light ABFT says *that* an error occurred, not *where*. Errors that cancel exactly (for example +5 in one element and −5 in another) are also missed.

Hardware cost for an N×N array (paper Table 3)	Traditional ABFT	Light ABFT	For N = 4
Extra adders	6N + 3	3N + 3	27 vs 15
Extra multipliers	2N + 1	1	9 vs 1
Can locate / correct the error	Yes / single errors	No / no	–

3. Project scope and paper coverage

We fully reproduced the paper's main contribution (Light ABFT) in hardware, and reproduced most of its experiments with our own fault-injection methods. We estimate that about **80–85% of the paper's content** is covered (our own estimate, weighting each section by its importance).

Sec.	Content	Our project	Where
2.2	Weight-stationary systolic array	Implemented (4×4, and 2×2/8×8 for scaling)	Sec. 4, 8.4
3	DSP-optimized PE: 2 multiplies per DSP, double-clocked DSP (Table 1)	Simplified: one multiplier per PE. Not reproduced	Sec. 12
4.1	Fault injection through ICAP on a Zynq	Replaced by RTL fault injection (board switch + simulation campaign)	Sec. 4.2, 7
5	FPGA fault model: error-pattern distribution (Table 2)	Reproduced with our campaign (Experiment 1)	Sec. 7.2
4.2, 5	CNN fault propagation, LeNet (Fig. 5)	Reproduced in Python (Experiment 4)	Sec. 9
5	CNN fault propagation, VGG16 (Fig. 6)	Not done (too heavy for a laptop)	Sec. 12

Sec.	Content	Our project	Where
6.1	Light ABFT technique and proof	Fully implemented in hardware and on the board	Sec. 2.3, 4, 6
6.2	Cost: Table 3, blocking model (Figs. 9, 10), CNN overhead	Measured in Vivado; Figs. 9–10 and CNN overhead reproduced (Exp. 2, 3, 5)	Sec. 8
7	Experimental validation: detection and false detections	Reproduced with a 10,880-run campaign	Sec. 7.1

Experiment	What it measures	Runs in	Paper item
Base design	Light ABFT works in simulation and on the board; resources, timing, power	Vivado + board	Sec. 6.1, 7
1. Fault campaign	Detection rate, false alarms, error-pattern distribution	Vivado simulation	Table 2, Sec. 7
2. Blocking model	Checking overhead vs. block size and error rate	Python	Figs. 9, 10
3. CNN overhead	Light ABFT extra operations in VGG16, ResNet18, ResNet50	Python	Sec. 6.2
4. LeNet faults	How MxM error patterns affect a neural network (PVF)	Python	Fig. 5
5. Scaling	Checker cost and timing for N = 2, 4, 8	Vivado (out-of-context)	Sec. 6 cost claims

4. Hardware design

All hardware is written in Verilog-2001 and built with Vivado 2024.2 for the EDGE Artix-7 board (XC7A35T-1FTG256, 50 MHz clock).

File	Function
pe.v	One processing element: sum_out = sum_in + a × b. Contains the fault-injection hooks.
systolic_4x4.v	16 PEs in a 4×4 grid; A flows right, partial sums flow down.
light_abft.v	Light ABFT: computes SV_A, SV_B, L (one multiplier, 4 cycles) and ΣC, then compares.
trad_abft.v	Traditional ABFT: 4 row checksums, 4 column checksums and the corner (9 multipliers); reports the faulty row/column.
abft_core.v	Test matrices, input skewing, control state machine, both checkers, capture of C for simulation.
top.v	Board wrapper: button/switch synchronizers and LED outputs.
tb_top.v, tb_campaign.v	Functional testbench and automated fault-injection campaign.
*_n.v, tb_scale.v	Parameterized N×N version used for the scaling experiment.

4.1 Test inputs

The base matrices are A = [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]] and B = [[1,0,2,1],[3,1,4,2],[2,2,1,0],[1,3,0,2]], so ΣC = L = 852. Four switches add a value 0–15 to every element, giving 16 different matrix pairs at runtime. Runtime inputs matter: with constant inputs, Vivado pre-computed the results and removed most of the array (constant propagation), leaving only 148 LUTs and 1 DSP.

4.2 Fault injection

Type	Fault model	Sites	Purpose
1	Bit flip in a PE's stored weight	16 PEs × 8 bits	Persistent upset, like a configuration fault
2	Multiplier output bit stuck-at-0	16 PEs × 16 bits	Logic fault in the multiplier
3	Multiplier output bit stuck-at-1	16 PEs × 16 bits	Logic fault in the multiplier
4	Bit flip in an SV_B register of the Light ABFT checker	4 registers × 10 bits	Faults in the checker itself (false detections)

On the board, switch SW7 enables a type 1 fault in bit 0 of PE(1,2), changing its stored weight from 4 to 5.

4.3 Board interface

Board item	Pin	Function
50 MHz oscillator	N11	System clock
Center / top push-button	M14 / K13	Reset / start a multiplication
SW0–SW3	L5, L4, M4, M2	Value added to every element of A and B
SW7	N1	Fault injection on/off
LED0 / LED1	J3 / H3	Done / OK (Light ABFT found no error)
LED2 / LED3	J1 / K1	Light ABFT error / Traditional ABFT error
LED4–LED7	L3, L2, K3, K2	Faulty column 0–3 reported by Traditional ABFT

5. How to reproduce this project (step by step)

This section lets someone new rebuild everything. All source files are in the project repository.

5.1 Board design in Vivado

1. Install AMD Vivado (ML Standard, free) with cable drivers. Create an **RTL project** for part **xc7a35tftg256-1**.
2. Add design sources: pe.v, systolic_4x4.v, light_abft.v, trad_abft.v, abft_core.v, top.v. Set **top** as top module.
3. Add simulation sources tb_top.v and tb_campaign.v, and the constraints file edge_artix7.xdc.
4. Set tb_top as simulation top, run **Behavioral Simulation**, type `run all` in the Tcl Console, and check that it prints ALL TESTS PASSED.
5. Run Synthesis, Implementation and Generate Bitstream. Open the reports: Utilization (Hierarchy view), Timing Summary, Power.
6. Connect the board by USB, open Hardware Manager → Auto Connect → Program Device.
7. Demo: press the center button (reset), then the top button (start). Flip SW7 and press start again.

5.2 Fault campaign (Experiment 1)

Set tb_campaign as simulation top, run Behavioral Simulation, then `run all`. The summary appears in the Tcl Console, and every run is saved to `fault_campaign.csv` in the project's `.sim/sim_1/behav/xsim` folder.

5.3 Scaling (Experiment 5)

In a separate project, add pe_n.v, systolic_n.v, light_abft_n.v, trad_abft_n.v, abft_core_n.v, scale_top.v and scale_ooc.xdc. Simulate tb_scale.v (change `parameter N` to 2, 4, 8), then set scale_top as top and run `source run_scaling.tcl` in the Tcl Console. It writes util_N*.txt, timing_N*.txt and power_N*.txt.

5.4 Python experiments (2, 3, 4)

```
python -m pip install -r requirements.txt
python exp2_blocking_model.py
python exp3_cnn_overhead.py
python exp4_lenet_pvf.py (first run downloads MNIST and trains LeNet)
```

6. Hardware results: simulation, board, Vivado reports

All results in this section are for the XC7A35T-1FTG256 at 50 MHz, using the board design (Section 4).

6.1 Functional simulation

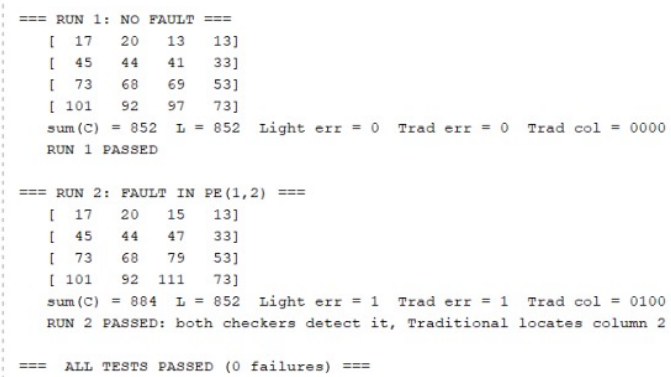


Figure 1. Vivado simulation output of tb_top. **What it shows:** without a fault, C is correct and ΣC = L = 852. With the SW7 fault, the whole of column 2 changes (13, 41, 69, 97 → 15, 47, 79, 111). ΣC becomes 884 while L stays 852, so Light ABFT flags the error, and Traditional ABFT flags column 2 (pattern 0100). The fault produced a full column error, matching the paper's fault model.

6.2 Hardware demonstration on the EDGE Artix-7 board

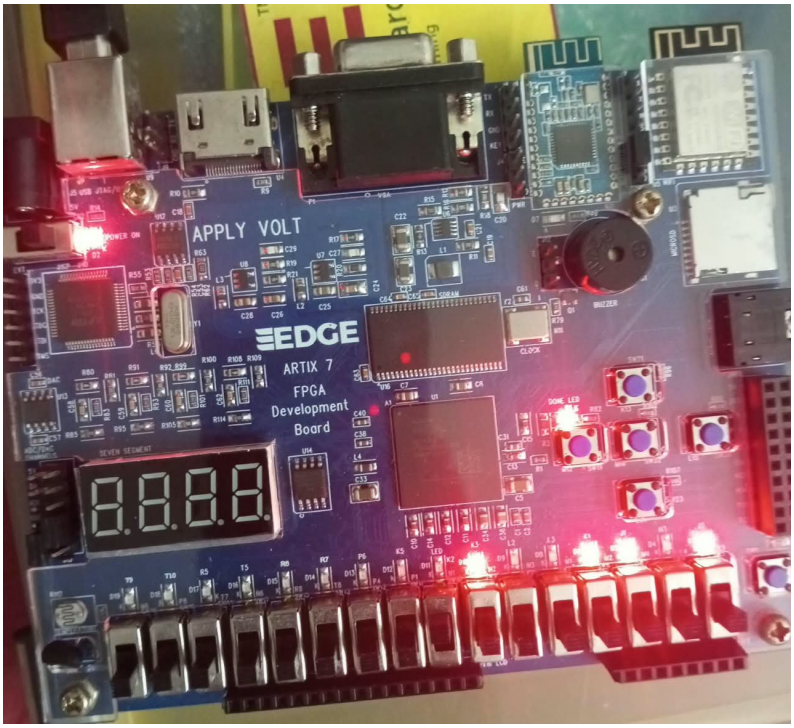


Figure 2. The design running on the EDGE Artix-7 board with the fault switch on. **What it shows:** LED0 (done), LED2 (Light ABFT error), LED3 (Traditional ABFT error) and LED6 (column 2) are lit, so both checkers detected the fault on real hardware and Traditional ABFT located it. The separate DONE LED near the FPGA only indicates successful configuration.

Board setting	LEDs on	Meaning
SW7 down, any SW0–SW3, start pressed	LED0, LED1	Multiplication done; ΣC = L, no error
SW7 up, start pressed	LED0, LED2, LED3, LED6	Error detected by both checkers; Traditional ABFT points to column 2

6.3 Resource utilization

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	DSPs (90)	Bonded IOB (170)	BUFGCTRL (32)
top	1190	646	409	1190	12	16	1
u_core (abft_core)	1190	638	406	1190	12	0	0
u_abft (light_abft	62	86	40	62	1	0	0
u_array (systolic_	731	279	262	731	0	0	0
u_trad (trad_abft)	382	254	178	382	11	0	0

Figure 3. Vivado utilization report (Hierarchy view) after implementation. **What it shows:** the whole design uses 1,190 of 20,800 LUTs (5.7%) and 12 of 90 DSPs. Light ABFT (u_abft) costs 62 LUTs and 1 DSP, while Traditional ABFT (u_trad) costs 382 LUTs and 11 DSPs.

6.4 Timing

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	8.524 ns	Worst Hold Slack (WHS):	0.116 ns	Worst Pulse Width Slack (WPWS):	9.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	1471	Total Number of Endpoints:	1471	Total Number of Endpoints:	657

All user specified timing constraints are met.

Figure 4. Timing summary. **What it shows:** all constraints are met at 50 MHz with +8.524 ns setup slack and +0.116 ns hold slack (0 failing endpoints out of 1,471). The critical path is 20 − 8.524 = 11.476 ns, so the design could run at about 87 MHz.

6.5 Power

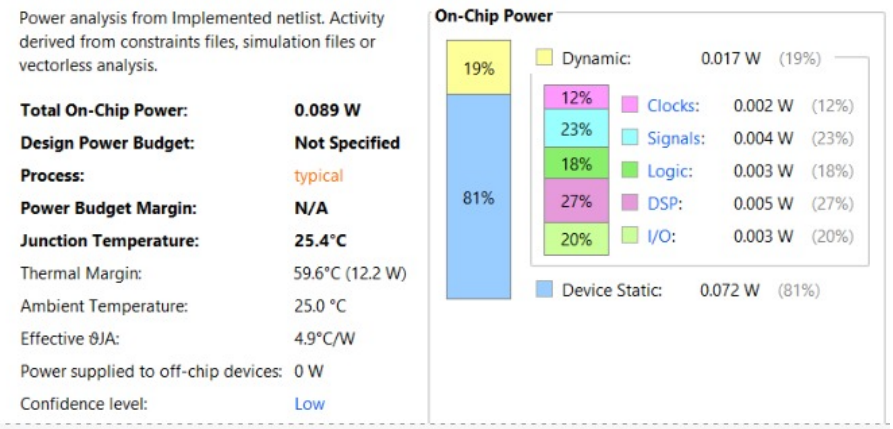


Figure 5. Power report. **What it shows:** total on-chip power is 0.089 W, of which 0.072 W is static chip leakage and only 0.017 W is caused by the design's activity (DSP 5 mW, signals 4 mW, logic 3 mW, I/O 3 mW, clocks 2 mW). This is Vivado's vectorless estimate (confidence: low).

6.6 Latency and throughput

Metric	Value
Cycles per 4×4 multiplication including the check	18 (1 load + 12 run + 4 checksum MAC + 1 compare)
Latency at 50 MHz	360 ns
Extra cycles added by Light ABFT	5
Peak array throughput	32 operations/cycle (16 PEs × multiply + add) = 1.6 GOPS at 50 MHz

7. Fault-injection campaign (Experiment 1)

The testbench injected every fault site from Section 4.2 under all 16 data patterns: 10,880 faulty runs plus 16 fault-free runs. For each run it compared C with a golden result and recorded what each checker reported.

```
Type 4: runs=640  C_wrong=0  DETECTED=0  MISSED=0  MASKED=0  FALSE_ALARM=640 | Trad detected=0  Trad located=0
------------------------------------------------------------
Array faults (types 1-3): runs with wrong C = 7329
  Light ABFT detection rate        = 100.00 %
  Traditional ABFT detection rate  = 100.00 %
  Traditional ABFT correct column  = 99.96 %
Light ABFT false-alarm rate (all 10880 fault runs) = 8.60 %
Light ABFT false alarms by type: 3 -> 296, 4 -> 640 ; Traditional false alarms: 3 -> 296, 4 -> 0
------------------------------------------------------------
ERROR PATTERN DISTRIBUTION (array faults, corrupted runs = 7329)
Pattern        Count    Ours     Paper (Table 2)
Single           339     4.63 %   27.07 %
Partial line    2031    27.71 %   43.28 %
Full line       4959    67.66 %   26.68 %
Full matrix        0     0.00 %    0.25 %
Random             0     0.00 %    2.72 %
  Type 1: single=0 partial=0 full_line=2048 full_matrix=0 random=0
  Type 2: single=150 partial=1035 full_line=296 full_matrix=0 random=0
  Type 3: single=189 partial=996 full_line=2615 full_matrix=0 random=0
Detailed results: fault_campaign.csv
------------------------------------------------------------
```

Figure 6. Campaign summary printed by tb_campaign in the Vivado Tcl Console (end of the output).

7.1 Detection and false detections

Metric	Light ABFT (ours)	Traditional ABFT (ours)	Paper (Light ABFT)
Runs with corrupted output (types 1–3)	7,329	7,329	–
Detection rate	**100%** (7,329 / 7,329)	100%	97.4% ± 0.15%
Missed errors (silent corruption)	0	0	2.6%

Metric	Light ABFT (ours)	Traditional ABFT (ours)	Paper (Light ABFT)
Correct faulty column identified	not possible	99.96% (7,326 / 7,329)	–
False detections, all 10,880 fault runs	8.60% (936)	2.72% (296)	2.39%
False detections, excluding checker faults	2.89% (296 / 10,240)	2.89% (296 / 10,240)	–

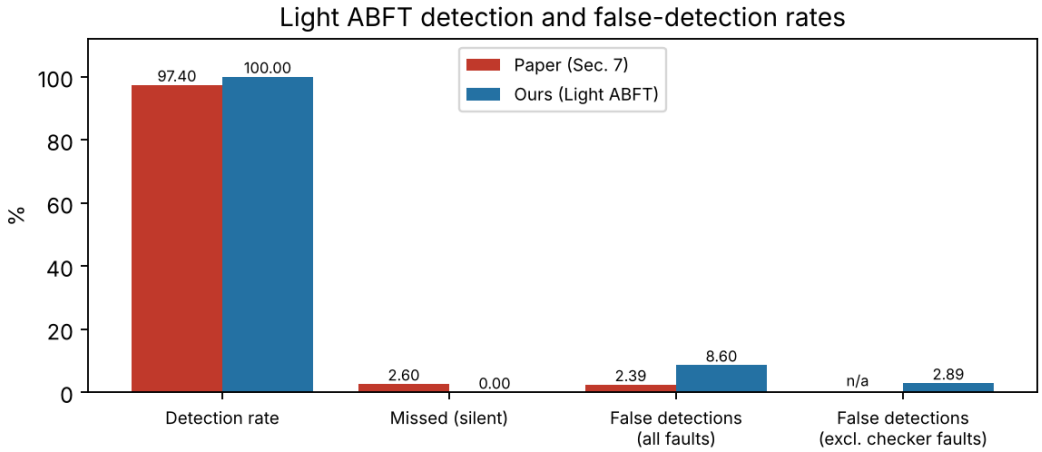


Figure 7. Detection and false-detection rates, paper vs. ours. **What it shows:** our Light ABFT detected every corrupted output (100% vs. the paper's 97.4%). Our false-detection rate is higher (8.6% vs. 2.39%) because 640 of our injections deliberately targeted the checker itself; without those, it is 2.89%, close to the paper.

7.2 Error-pattern distribution (paper Table 2)

Pattern	Ours: count	Ours: %	Paper: %
Single	339	4.63	27.07
Partial line	2,031	27.71	43.28
Full line	4,959	67.66	26.68
Full matrix	0	0.00	0.25
Random	0	0.00	2.72
Line errors (partial + full)	**6,990**	**95.37**	**69.96**

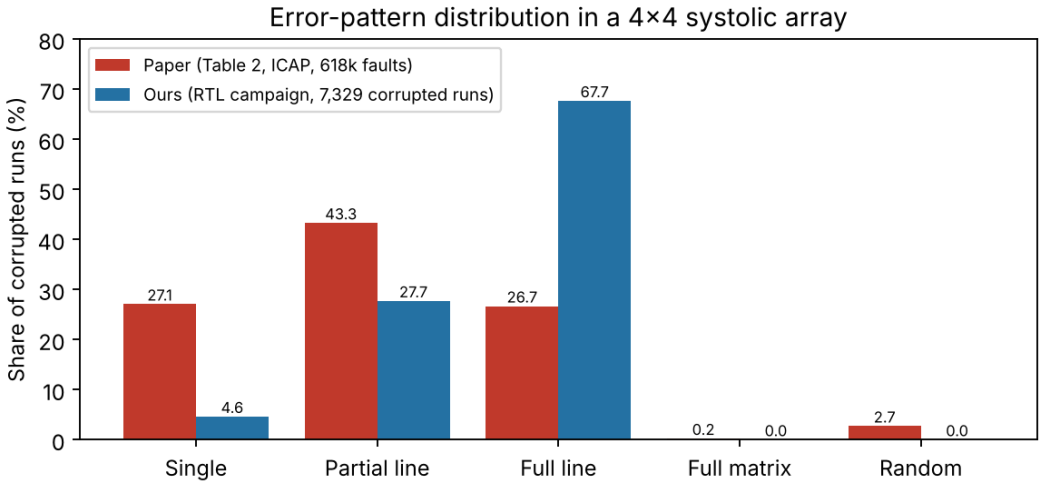


Figure 8. Error-pattern distribution, paper vs. ours. **What it shows:** in both, line errors dominate (95.4% for us, 70.0% in the paper), which is the paper's main argument for detection-only checking. Our share is higher and we see no random or full-matrix errors, because our faults sit inside one PE and can only disturb one column, while the paper's configuration faults also hit routing and control logic, which can spread errors more widely.

8. Cost analysis (Experiments 2, 3 and 5)

8.1 Light ABFT vs. Traditional ABFT on the board design

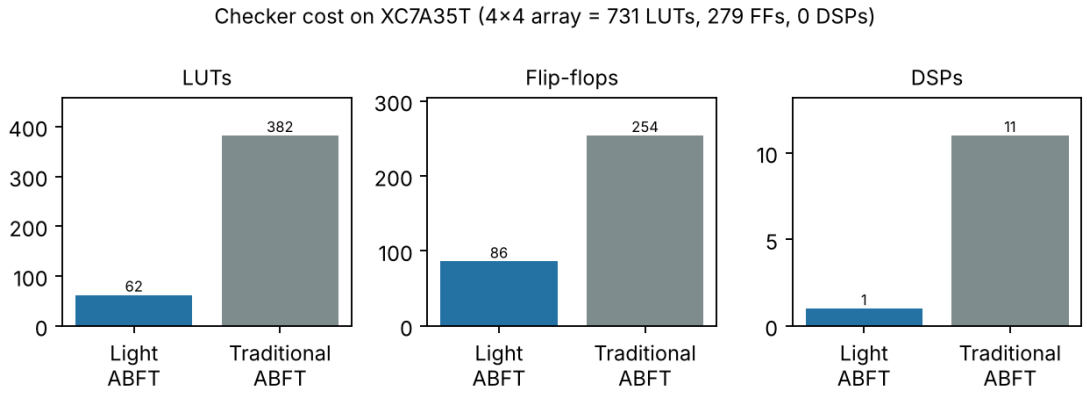


Figure 9. Measured checker cost. **What it shows:** for the same 100% detection rate, Traditional ABFT needs 6.2× the LUTs, 3.0× the flip-flops and 11× the DSPs of Light ABFT. Light ABFT costs 8.5% of the array's LUTs; Traditional ABFT costs 52.3%. The extra hardware only buys the ability to locate the faulty column.

8.2 Overhead vs. block size (Experiment 2, paper Figs. 9 and 10)

Large matrices are processed in B×B blocks. Checking each block costs more arithmetic than checking the whole matrix once, but after an error only one block must be recomputed. The paper's Equation 14 combines both effects, with R = expected number of executions between errors. Our script evaluates the same equation with the Light ABFT costs from Table 3 (it first checks the paper's worked example, H(4) = 53 additions and 4 multiplications).

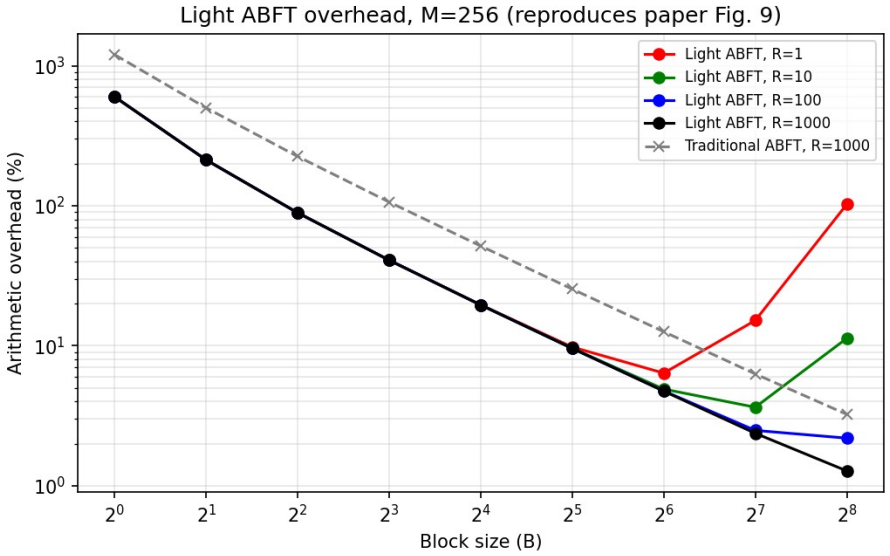


Figure 10. Our reproduction of the paper's Fig. 9 (M = 256). **What it shows:** overhead falls steeply as blocks grow, until recomputation costs dominate. With frequent errors (R = 1) the optimum is B = 64 (6.4% overhead); with rare errors (R = 1000) it moves to B = 256 (1.3%). Traditional ABFT (grey) is about twice as expensive at every block size.

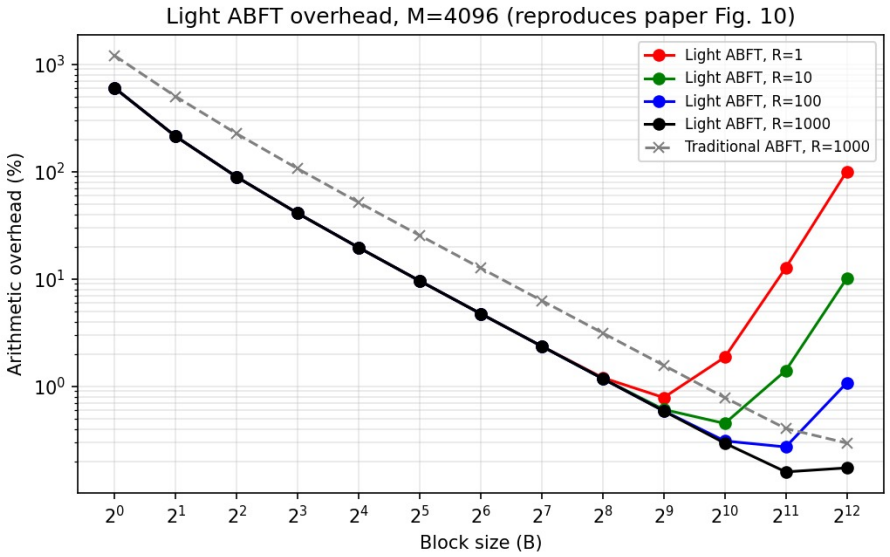


Figure 11. Our reproduction of the paper's Fig. 10 (M = 4096). **What it shows:** for very large matrices the minimum overhead drops below 1% (0.78% at R = 1, 0.16% at R = 1000). As in the paper, the higher the error rate, the earlier the curve turns upward.

Comparison with the paper: both figures follow the paper's equation, and show the same behaviour the paper describes: each R has its own optimal block size, and that optimum appears at smaller blocks when errors are more frequent. To compare visually, place the paper's Figs. 9 and 10 next to Figures 10 and 11.

8.3 Overhead in real CNNs (Experiment 3, paper Section 6.2)

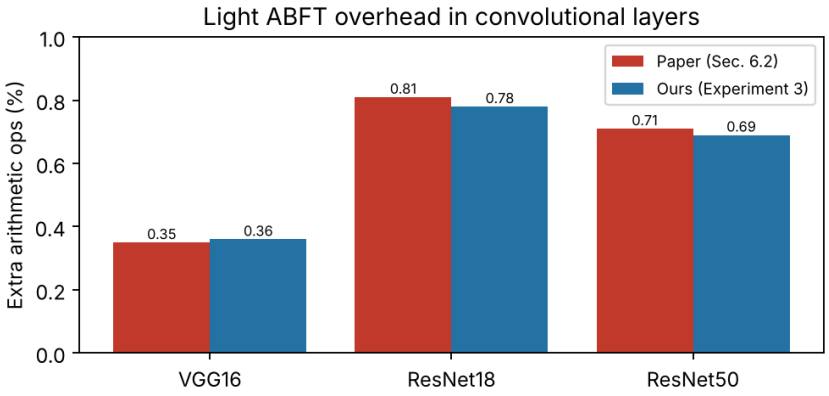


Figure 12. Light ABFT arithmetic overhead in the convolutional layers of three CNNs. **What it shows:** our values (0.36%, 0.78%, 0.69%) are within 0.03 percentage points of the paper's (0.35%, 0.81%, 0.71%). Protecting a whole CNN with Light ABFT costs less than 1% extra arithmetic.

8.4 Scaling with array size (Experiment 5)

A parameterized version of the design was verified in simulation for N = 2, 4 and 8 (20 random matrix pairs and one fault in every PE, all tests passed, Figure 13) and implemented out-of-context at a 100 MHz target.

```
run all
N=2 Part 1 (no fault, 20 tests): PASSED
N=2 Part 2 (fault in each of 4 PEs, 0 masked): PASSED
=== N=2:  ALL TESTS PASSED (0 failures) ===
$finish called at time : 2920 ns : File "C:/Users/hp/abft_scale_version/tb_scale.v" Line 89
run all
N=4 Part 1 (no fault, 20 tests): PASSED
N=4 Part 2 (fault in each of 16 PEs, 0 masked): PASSED
=== N=4:  ALL TESTS PASSED (0 failures) ===
$finish called at time : 7240 ns : File "C:/Users/hp/abft_scale_version/tb_scale.v" Line 89
```

```
run all
N=8 Part 1 (no fault, 20 tests): PASSED
N=8 Part 2 (fault in each of 64 PEs, 0 masked): PASSED
=== N=8:  ALL TESTS PASSED (0 failures) ===
$finish called at time : 30280 ns : File "C:/Users/hp/abft_scale_version/tb_scale.v" Line 89
```

Figure 13. tb_scale results for N = 2, 4 and 8: all tests passed.

N	Array LUTs	Light LUTs (% of array)	Trad LUTs (% of array)	Light / Trad DSPs	WNS at 100 MHz	Est. Fmax
2	438	161 (36.8%)	748 (170.8%)	0 / 0	+0.337 ns	≈103 MHz
4	1,630	133 (8.2%)	444 (27.2%)	1 / 11	+0.277 ns	≈103 MHz
8	6,549	440 (6.7%)	1,553 (23.7%)	1 / 21	−5.279 ns	≈65 MHz

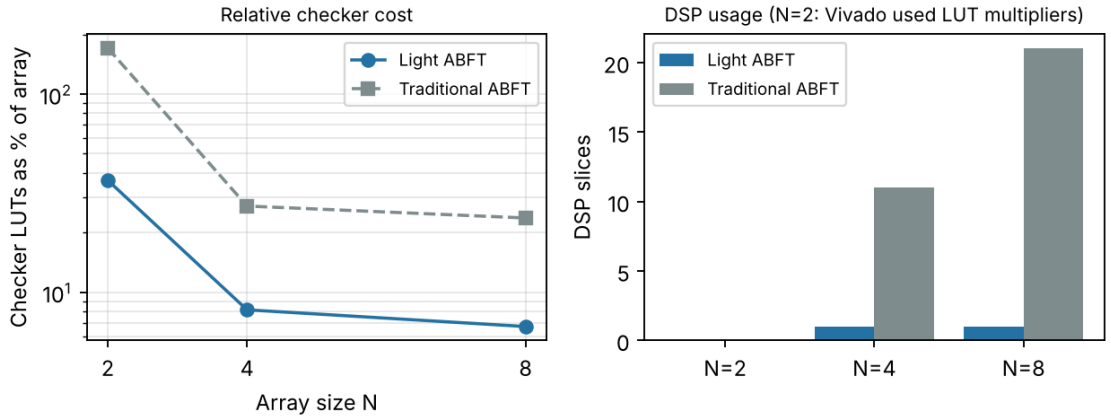


Figure 14. Checker cost vs. array size. **What it shows:** Light ABFT's share of the array falls from 36.8% (N=2) to 8.2% (N=4) and 6.7% (N=8), confirming the paper's claim that it becomes relatively cheaper as arrays grow. Traditional ABFT stays 3–5× more expensive at every size and needs up to 21 DSPs vs. 1.

Notes: at N = 2, Vivado built all multipliers from LUTs (0 DSPs), which inflates the LUT counts at that size. At N = 8 the design misses 100 MHz (about 65 MHz achievable); the long path is likely the wide checksum addition, which could be pipelined. These numbers use full-range random 8-bit inputs, so the N = 4 values differ from the board design in Section 6.

9. Effect on a neural network (Experiment 4)

Following the paper's Section 4.2, we ran LeNet (trained on MNIST) in Python with every convolutional and fully-connected layer computed as a matrix multiplication split into N×N blocks, as a systolic array would. A persistent error pattern (single, partial line, full line, full matrix) was injected into every block of a target layer and all later layers. For each combination, 200 faults were injected. We measured the **Program Vulnerability Factor (PVF)**: *tolerable* = output scores changed but the predicted digit stayed the same; *critical* = the predicted digit changed.

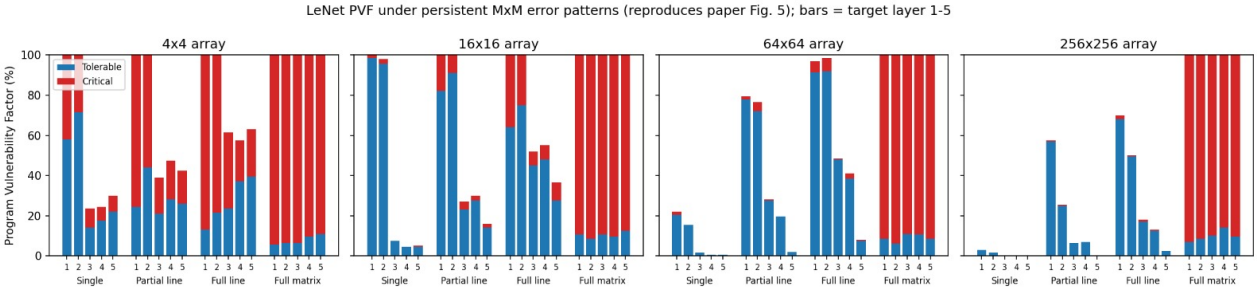


Figure 15. Our reproduction of the paper's Fig. 5 (LeNet PVF; bars = target layer 1–5; blue = tolerable, red = critical). **What it shows:** (1) larger arrays are less vulnerable: a single-element error in layer 1 affects 100% of outputs on a 4×4 array but only 3% on a 256×256 array, because a small array repeats the faulty block many times across a layer; (2) more severe patterns cause more errors: full-matrix errors are about 90% critical at every size; (3) faults that start in earlier layers cause more errors than faults in later layers.

10. Paper vs. ours: summary comparison

Item	Paper	Ours	Agreement
Device / array	XC7Z020 (Zynq), 14×14 at 200 MHz; 4×4 for faults	XC7A35T (Artix-7), 4×4 at 50 MHz (≈87 MHz max)	Different setup
Operations per cycle	$8N^2$ (4 multiplies per DSP)	$2N^2$ (1 multiply per PE)	Simplified PE
Fault injection	ICAP, 623,000 configuration faults	RTL models, 10,880 faults + board switch	Different method
Light ABFT detection rate	97.4%	100%	Similar (ours higher)
False detections	2.39%	8.60% (2.89% excl. checker faults)	Close when comparable
Line errors (Table 2)	69.96%	95.37%	Same conclusion
Extra multipliers, N = 4	1 vs 9 (formula)	1 vs 9 (built); 62 vs 382 LUTs measured	Matches
Overhead vs. block size (Figs. 9, 10)	U-shaped curves, optimum shifts with R	Same equation, same shape	Matches
CNN overhead VGG16 / ResNet18 / ResNet50	0.35% / 0.81% / 0.71%	0.36% / 0.78% / 0.69%	Matches (±0.03 pp)
LeNet Fig. 5 trends	Larger array → lower PVF; severity and early layers → more critical	Same trends	Trends match
Checker overhead vs. array size	Claimed to shrink with N	36.8% → 8.2% → 6.7% (N = 2, 4, 8)	Confirmed

11. Our contribution

1. **Independent, readable RTL** of a weight-stationary systolic array with Light ABFT, written from the paper rather than taken from the authors' DSP-optimized generator.
2. **Working hardware demonstration on a plain Artix-7 board**, without the Zynq processor the paper relied on, using switches for data and fault injection and LEDs for results.
3. **A side-by-side Traditional ABFT implementation** on the same array, giving a measured cost comparison (6.2× LUTs, 11× DSPs) and a visible demonstration of error localization (LED4–LED7).
4. **An RTL fault-injection framework and automated campaign**: four fault types, any PE and bit, 10,880 runs, CSV logging, and automatic error-pattern classification.
5. **A parameterized N×N design** and scripted scaling study (N = 2, 4, 8), measuring how checker cost changes with size.
6. **Reproduction of the paper's analytical and CNN experiments** (Figs. 5, 9, 10 and the CNN overhead values) in Python.
7. **Practical observations:** constant test inputs let synthesis delete most of the array, so runtime inputs are needed for valid resource numbers; checking during idle cycles lets the checker flag faulty hardware even when the valid outputs are correct.

12. Limitations and future work

Limitation	Possible next step
RTL fault models; no configuration-memory, routing or control faults	Inject real configuration upsets with AMD's Soft Error Mitigation (SEM) IP, which supports 7-series devices

Limitation	Possible next step
Simplified PE (no DSP packing or double-clocked DSPs, paper Table 1)	Run the authors' LABFT generator for an 8×8 array (64 DSPs fits the XC7A35T) and compare GOPS
VGG16 fault experiment (paper Fig. 6) not done	Run a reduced version (a few layers, fewer faults) on a GPU
Corruption values in the CNN experiment are an assumption	Derive corruption values from the actual bit-level errors seen in the hardware campaign
N = 8 misses 100 MHz	Pipeline the checksum adder trees
Power is a vectorless estimate	Use simulation switching activity (SAIF) for a higher-confidence estimate

13. Conclusion

We implemented the paper's Light ABFT error detection on a systolic array, verified it in simulation and demonstrated it on a physical Artix-7 FPGA. In 10,880 fault-injection runs it detected every corrupted output, matching a traditional ABFT checker that needed 6.2× the LUTs and 11× the DSPs. Its CNN overhead and blocking-model behaviour match the paper closely, line errors dominate in our fault model as in the paper's, and its relative cost falls as arrays grow. These results support the paper's conclusion: on FPGA systolic arrays, where faults persist until reconfiguration, cheap detection is a better trade-off than expensive correction.

References

1 F. Libano, P. Rech and J. Brunhaver, "Efficient Error Detection for Matrix Multiplication With Systolic Arrays on FPGAs," IEEE Transactions on Computers, vol. 72, no. 8, pp. 2390–2403, 2023. DOI: 10.1109/TC.2023.3248282.

2 F. Libano, LABFT: parametric RTL generator for systolic arrays with error detection, https://github.com/lllibano/LABFT

3 K.-H. Huang and J. A. Abraham, "Algorithm-Based Fault Tolerance for Matrix Operations," IEEE Transactions on Computers, vol. C-33, no. 6, pp. 518–528, 1984.

4 Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

5 EDGE Artix 7 FPGA Development Board, user manual and pin details, allaboutfpga.com.

6 AMD, Vivado Design Suite 2024.2 and 7 Series DSP48E1 Slice User Guide (UG479).